디지털논리회로설계

# VHDL 과제 3 결과 보고서

| | |
|---|---|
| **담당교수** | 장범진 교수님 |
| **학과** | 로봇공학과 |
| **학번** | 2023010064 |
| **이름** | 임주언 |
| **제출일** | 2026. 11. 10. |

## 0. 실습 환경

이번 과제에서는 RS 래치, D 래치, D 플립플롭, JK 플립플롭, set/clr가 있는 D 플립플롭을 만들었다. 각 회로를 Logisim에서 게이트로 직접 그린 것과 VHDL로 작성한 것 두 가지로 구현하였다. 사용한 프로그램은 다음과 같다.

- Logisim-evolution 5.0.0 : 게이트 회로 작성(내장 래치/플립플롭 부품은 사용하지 않음), VHDL Entity 시뮬레이션
- Questa FPGA Starter Edition : VHDL 컴파일(vcom), 테스트벤치 시뮬레이션(vsim)
- Quartus Prime Lite 25.1 : 합성(Analysis & Synthesis), RTL Viewer로 회로도 확인. 디바이스는 MAX 10 (10M08SAE144C8G)으로 설정하였다.

게이트 회로는 수업자료(9-1lec)와 실습자료(VHDL_2), 11/11 공지에 나온 회로 형태를 기준으로 그렸다. Logisim 파일(assignment3.circ)에는 문제마다 게이트로 그린 시트(pN_…_gate)와 VHDL을 불러온 시트(pN_…_vhdl)를 따로 만들었다. 출력은 모두 Q와 Qn이고, 플립플롭은 상승 에지에서 동작하도록 했다. clk는 입력 핀으로 두고 직접 0에서 1로 바꿔서 상승 에지를 만들었다.

## 1. RS 래치

NOR 게이트 2개의 출력을 서로의 입력으로 연결하였다. 식으로 쓰면 Q = (R + Qn)′, Qn = (S + Q)′ 이다. NOR는 입력에 1이 하나라도 있으면 0이 나오기 때문에 S=1을 넣으면 Qn이 0이 되고 Q는 1이 된다. S=R=0이면 두 게이트가 서로 값을 잡고 있어서 이전 상태가 유지된다.

| S | R | Q+ | Qn+ | 동작 |
|---|---|---|---|---|
| 0 | 0 | Q | Qn | 유지 |
| 0 | 1 | 0 | 1 | Reset |
| 1 | 0 | 1 | 0 | Set |
| 1 | 1 | 0 | 0 | 금지 |

특성식은 Q+ = S + R′Q (SR=0) 이다. S=R=1을 넣으면 Q와 Qn이 둘 다 0이 된다. 여기서 S, R을 동시에 0으로 내리면 진동이 생겨서 Logisim에서 선이 빨갛게 변했다. 그래서 테스트할 때는 금지 입력 다음에 Set을 넣어서 빠져나왔다.

### (1) Logisim 게이트 회로

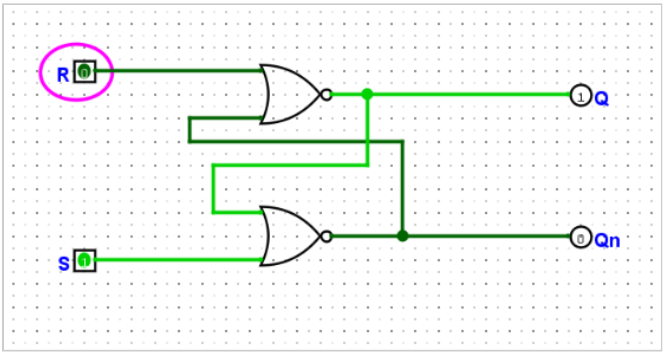



그림 1. Set (S=1, R=0)

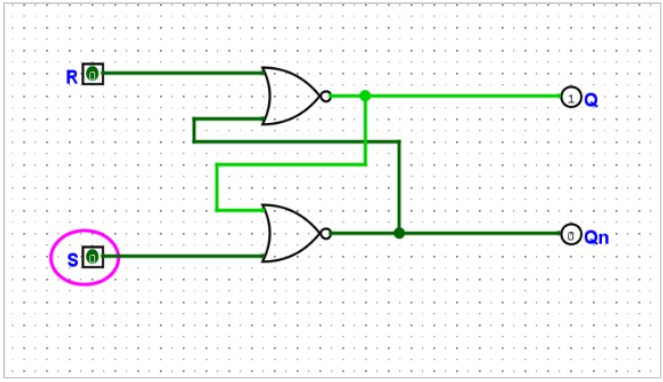



그림 2. Hold, Q=1 유지 (S=R=0)

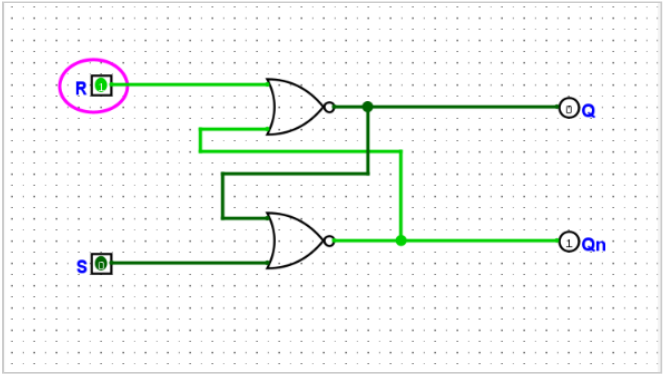



그림 3. Reset (S=0, R=1)

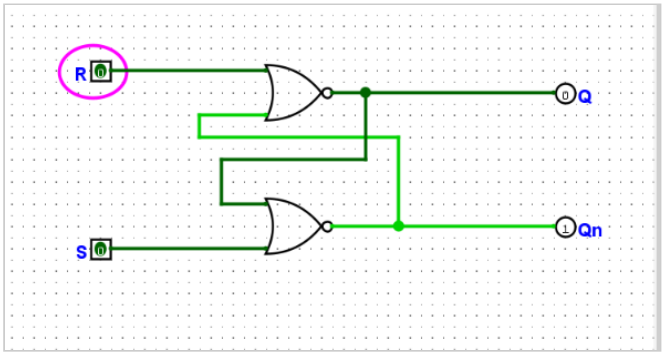



그림 4. Hold, Q=0 유지 (S=R=0)

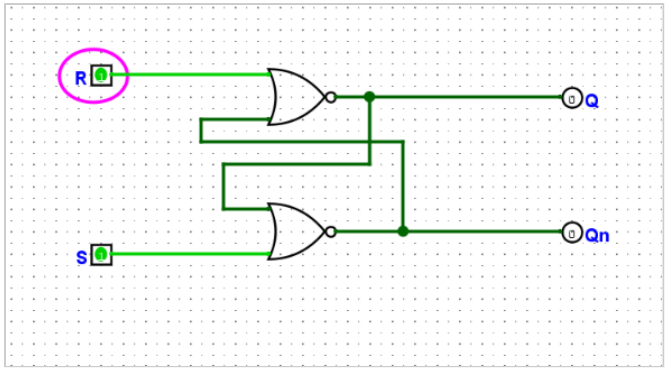



그림 5. S=R=1 입력 시 Q=Qn=0

### (2) VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- RS latch : two cross-coupled NOR gates
entity rs_latch is
    port(
        s, r  : in  std_logic;
        q, qn : out std_logic
    );
end entity rs_latch;

architecture bhv of rs_latch is
    signal q_i  : std_logic := '0';
    signal qn_i : std_logic := '1';
begin
    q_i  <= r nor qn_i;   -- upper NOR : Q  = (R + Q')'
    qn_i <= s nor q_i;    -- lower NOR : Q' = (S + Q)'

    q  <= q_i;
    qn <= qn_i;
end architecture bhv;
```

게이트 회로와 같은 식을 그대로 썼다. out 포트는 내부에서 읽을 수 없어서 q_i, qn_i 신호를 따로 만들어 되먹임하였다. 아래는 이 파일을 Logisim에 VHDL Entity로 불러와서 같은 입력을 넣어 본 결과이다.

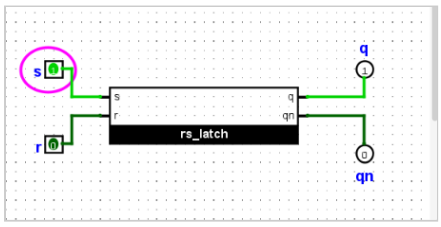



그림 6. Set (S=1, R=0) (VHDL)

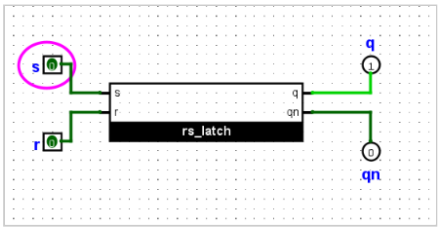



그림 7. Hold, Q=1 유지 (S=R=0) (VHDL)

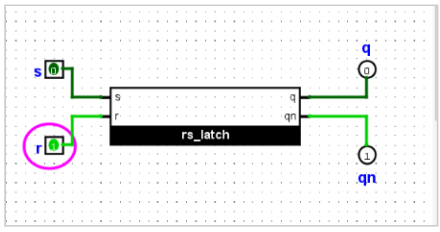



그림 8. Reset (S=0, R=1) (VHDL)

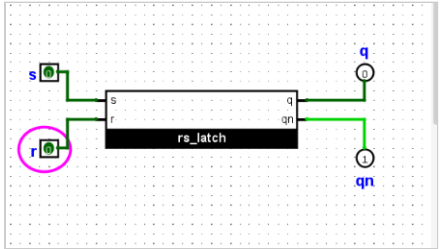



그림 9. Hold, Q=0 유지 (S=R=0) (VHDL)

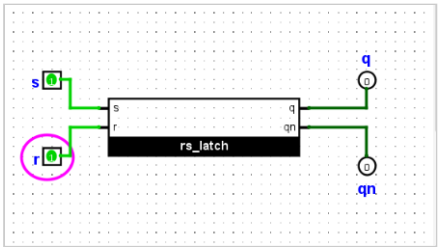



그림 10. S=R=1 입력 시 Q=Qn=0 (VHDL)

### (3) 합성 결과

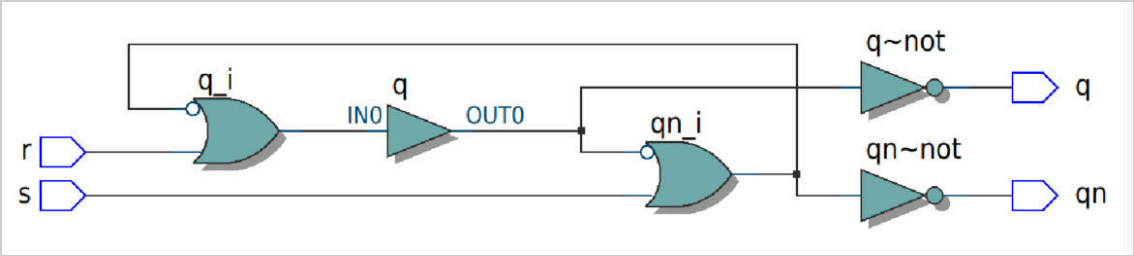



그림 11. p1 RTL Viewer 회로도

### (4) 시뮬레이션 결과

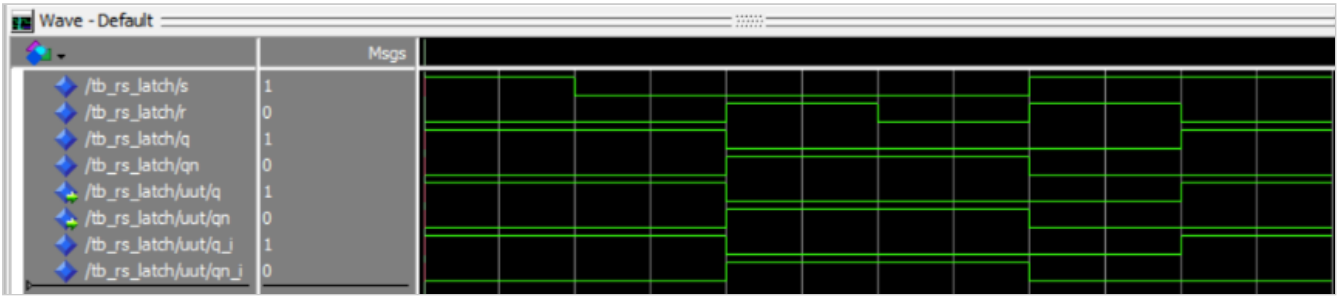



그림 12. tb_rs_latch 시뮬레이션 파형 (Questa)

Set, 유지, Reset, 유지, S=R=1, Set 순서로 10 ns마다 입력을 바꾸었다. S=R=1 구간에서 q와 qn이 모두 0이 되는 것까지 표와 같게 나왔다. 합성 회로도를 보면 q_i, qn_i가 OR 게이트(입력 하나 반전) 두 개로 서로 엇갈려 연결되어 있는데, 게이트로 그린 회로와 같은 구조이다.

## 2. D 래치

RS 래치에서 S=D, R=D′로 묶으면 금지 입력이 생기지 않는다. 실습자료(VHDL_2 p.3)와 수업자료(9-1lec p.16)의 회로대로 NAND 4개와 NOT 1개로 만들었다. 앞쪽 G1 = (D·en)′, G2 = (D′·en)′ 이고 D′는 NOT으로 만들었다. 뒤쪽 G3, G4는 NAND RS 래치이다.

en=1이면 G1=D′, G2=D가 되어 Q가 D를 그대로 따라가고, en=0이면 G1, G2가 둘 다 1이 되어서 값이 유지된다. 특성식은 Q+ = en·D + en′·Q 이다.

| en | D | Q+ |
|---|---|---|
| 1 | 0 | 0 |
| 1 | 1 | 1 |
| 0 | x | Q (유지) |

### (1) Logisim 게이트 회로

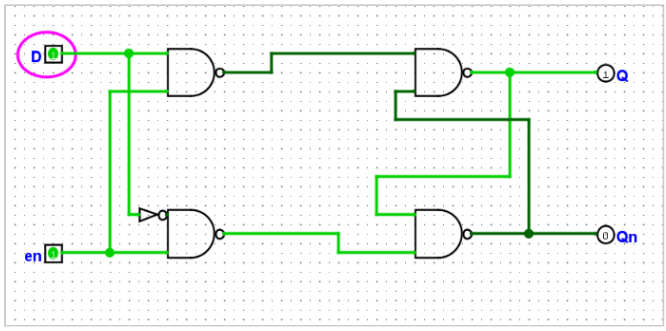



그림 13. en=1, D=1 일 때 Q=1

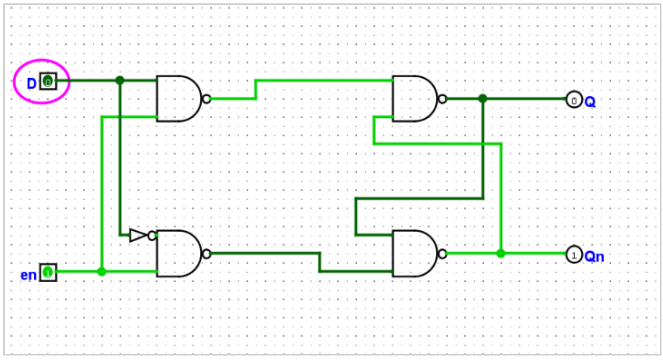



그림 14. en=1, D=0 일 때 Q=0

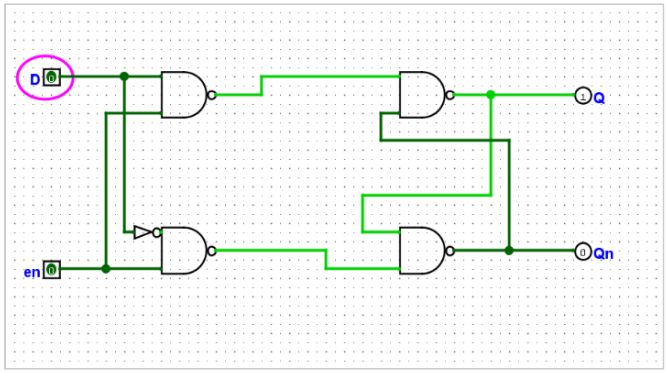



그림 15. en=0 에서 D 변경, Q=1 유지

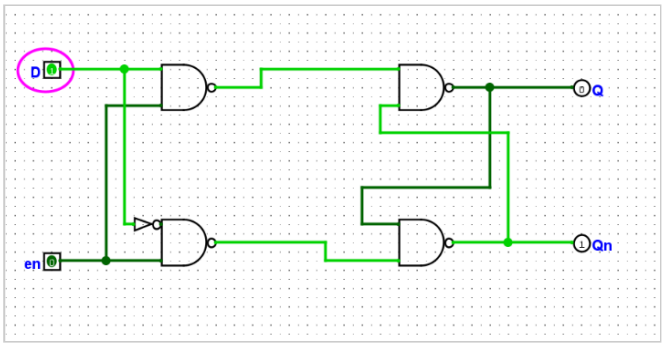



그림 16. en=0 에서 D 변경, Q=0 유지

### (2) VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- D latch with enable (level triggered)
entity d_latch is
    port(
        d, en : in  std_logic;
        q, qn : out std_logic
    );
end entity d_latch;

architecture bhv of d_latch is
    signal q_i : std_logic := '0';
begin
    process(d, en)
    begin
        if en = '1' then
            q_i <= d;        -- en=1 : transparent, Q follows D
        end if;              -- en=0 : no assignment -> keeps old value
    end process;

    q  <= q_i;
    qn <= not q_i;
end architecture bhv;
```

en='1'일 때만 q_i에 d를 넣고 else는 쓰지 않았다. en='0'이면 대입이 없어서 값이 남아 있게 되고, 이 부분이 래치가 된다.

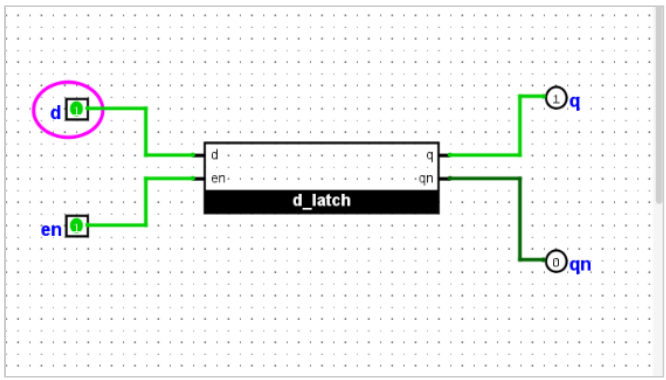



그림 17. en=1, D=1 일 때 Q=1 (VHDL)

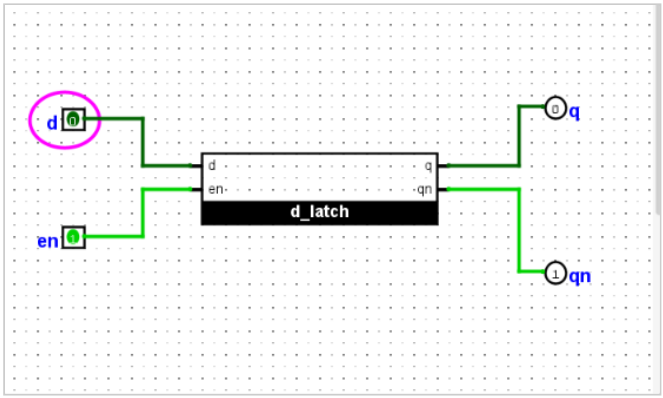



그림 18. en=1, D=0 일 때 Q=0 (VHDL)

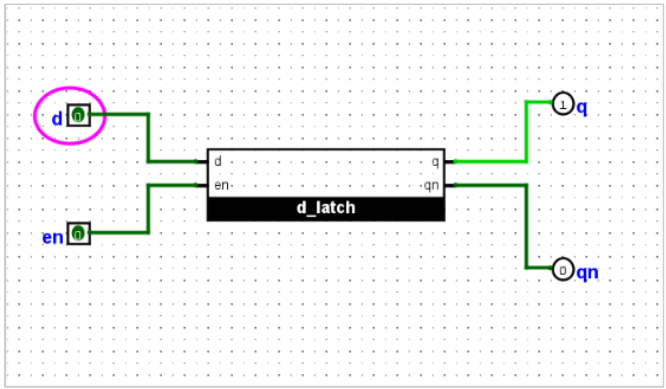



그림 19. en=0 에서 D 변경, Q=1 유지 (VHDL)

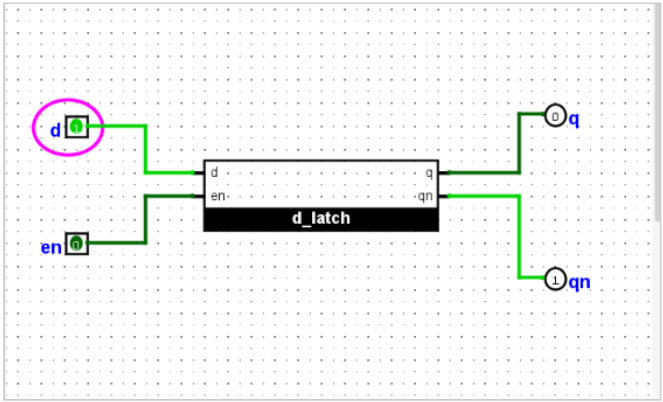



그림 20. en=0 에서 D 변경, Q=0 유지 (VHDL)

### (3) 합성 결과

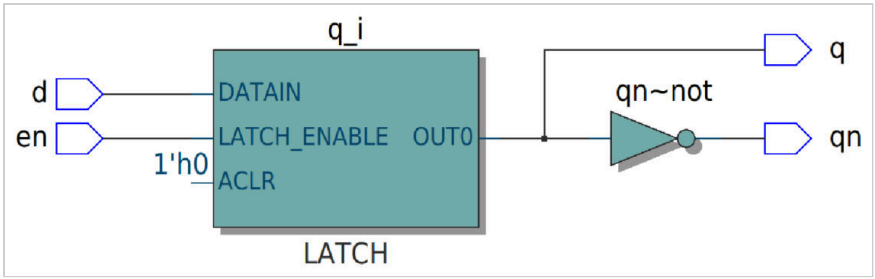



그림 21. p2 RTL Viewer 회로도

### (4) 시뮬레이션 결과

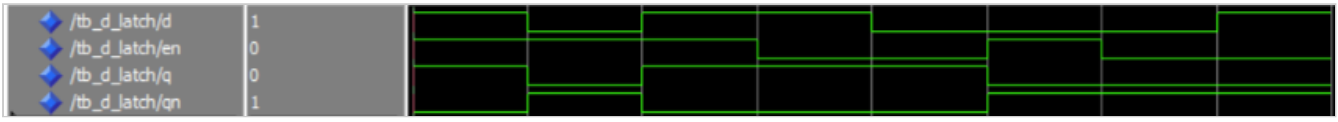



그림 22. tb_d_latch 시뮬레이션 파형 (Questa)

en=1인 구간에서는 q가 d를 따라가고, en=0으로 바꾼 뒤에는 d를 바꿔도 q가 변하지 않았다. 합성 결과도 LATCH 하나가 나왔다(DATAIN=d, LATCH_ENABLE=en). Quartus에서 "inferring latch" 경고가 나오는데, 래치를 만드는 문제라서 의도한 결과이다.

## 3. D 플립플롭 (상승 에지)

실습자료(VHDL_2 p.8)의 Master-Slave 회로대로, 2번의 D 래치를 두 개 이어서 만들었다. G1~G4가 master, G5~G8이 slave이고, 각 래치 안에 D를 뒤집는 NOT이 하나씩 있다. slave의 D 입력은 master의 출력(G3)이다. clk는 NOT을 한 번 거쳐 master의 en으로 들어가고, 그 신호를 NOT으로 한 번 더 뒤집어 slave의 en으로 넣었다. 결국 master는 clk′, slave는 clk로 동작한다.

clk=0일 때는 master가 열려 D를 받고 slave는 닫혀서 Q가 유지된다. clk가 1로 올라가면 master가 닫히면서 그 순간의 D를 잡고, slave가 열리면서 그 값이 Q로 나온다. clk=1인 동안은 master가 닫혀 있어서 D가 바뀌어도 Q에 영향이 없다. 그래서 Q는 상승 에지에서만 바뀐다(Q+ = D).

처음 수정했을 때 새로 그린 선이 회색으로 나오고 출력이 E로 떠서, 시뮬레이션을 리셋(Ctrl+R)하고 D=0에서 clk를 한 번 움직여 초기값을 정한 뒤 테스트하였다.

### (1) Logisim 게이트 회로

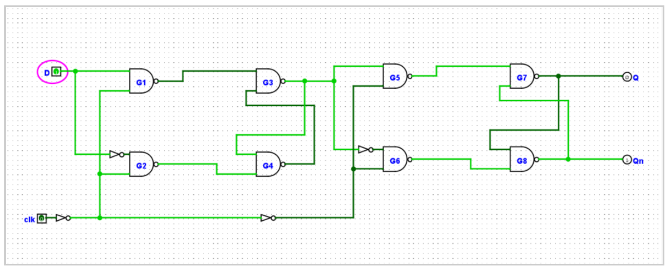



그림 23. clk=0, D=1 (에지 전)

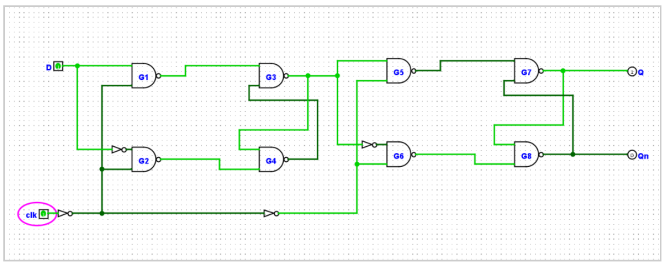



그림 24. clk 상승 에지 후 Q=1

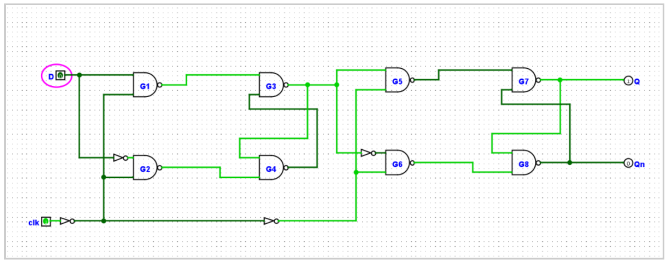



그림 25. clk=1 상태에서 D 변경

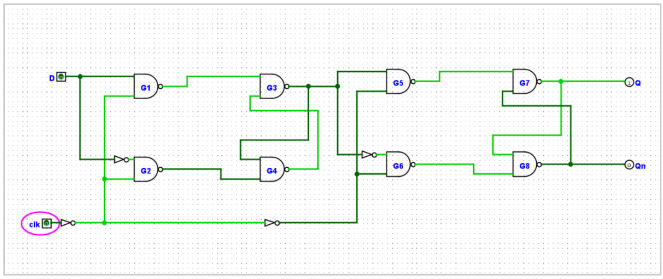



그림 26. clk 하강 에지

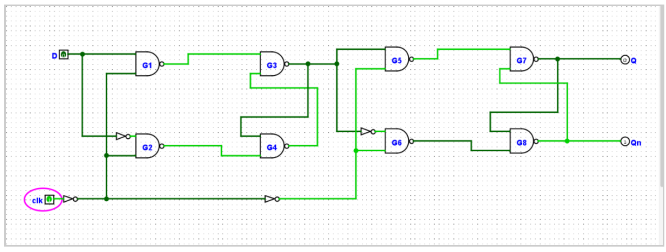



그림 27. 두 번째 상승 에지

### (2) VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- Positive-edge-triggered D flip-flop
entity d_ff is
    port(
        d, clk : in  std_logic;
        q, qn  : out std_logic
    );
end entity d_ff;

architecture bhv of d_ff is
    signal q_i : std_logic := '0';
begin
    process(clk)
    begin
        if rising_edge(clk) then   -- only at the 0 -> 1 moment of clk
            q_i <= d;
        end if;
    end process;

    q  <= q_i;
    qn <= not q_i;
end architecture bhv;
```

rising_edge(clk)일 때만 q_i에 d를 넣었다. process의 감도 목록에는 clk만 넣었다.

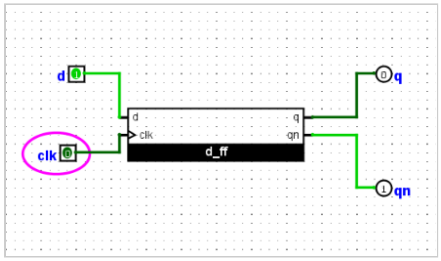



그림 28. clk=0, D=1 (에지 전) (VHDL)

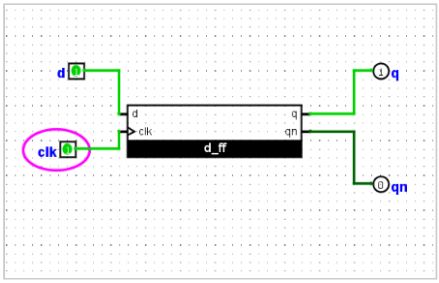



그림 29. clk 상승 에지 후 Q=1 (VHDL)

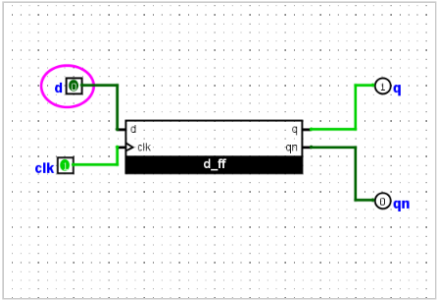



그림 30. clk=1 상태에서 D 변경 (VHDL)

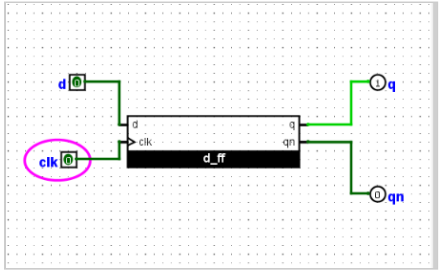



그림 31. clk 하강 에지 (VHDL)

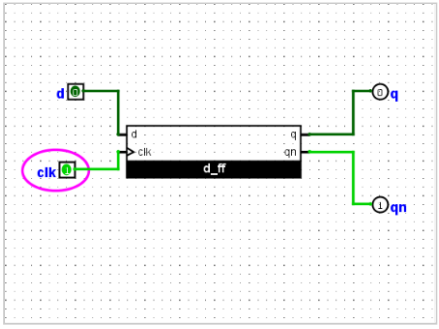



그림 32. 두 번째 상승 에지 (VHDL)

### (3) 합성 결과

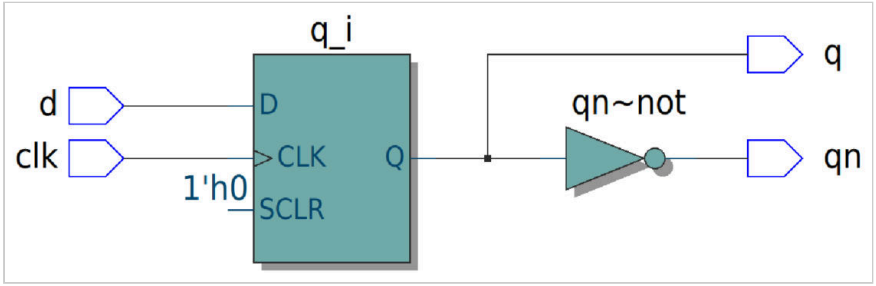



그림 33. p3 RTL Viewer 회로도

### (4) 시뮬레이션 결과

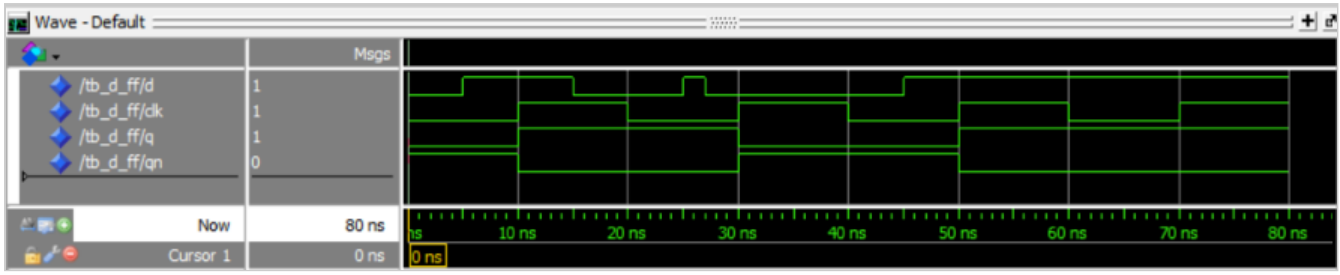



그림 34. tb_d_ff 시뮬레이션 파형 (Questa)

클럭 주기는 20 ns이고 상승 에지는 10, 30, 50, 70 ns이다. 15 ns(clk=1일 때)와 25 ns(짧은 펄스)에 d를 바꿨는데 q는 바뀌지 않았고, 에지 순간의 d 값만 q에 들어갔다. 합성하면 DFF 하나가 나온다.

## 4. JK 플립플롭

JK 플립플롭은 RS의 금지 입력(J=K=1)을 토글로 바꾼 것이다. 출력을 입력쪽으로 되먹임해서 J는 Qn과, K는 Q와 AND되게 했다. 그래서 Q=0일 때는 J만, Q=1일 때는 K만 의미가 있다. 특성식은 Q+ = JQ′ + K′Q 이다.

| J | K | Q+ | 동작 |
|---|---|---|---|
| 0 | 0 | Q | 유지 |
| 0 | 1 | 0 | Reset |
| 1 | 0 | 1 | Set |
| 1 | 1 | Q′ | Toggle |

앞부분은 11/11 공지에 나온 JK 회로(VHDL_2 p.11과 같은 형태)를 그대로 master로 사용하였다. G1(A) = (J·Qn·clk)′, G2(D) = (K·Q·clk)′ 는 3입력 NAND이고 G3(B), G4(C)는 교차 결합 NAND이다. 공지에 적힌 대로 이 회로 하나만으로는 J=K=1에서 출력이 계속 반전되며 발진하기 때문에, 뒤에 SR 래치(G5~G8)를 붙여 master-slave 구조로 만들었다(9-1lec p.20, VHDL_2 p.12). 되먹임은 master가 아니라 최종 출력 Q, Qn에서 가져왔다.

클럭은 3번처럼 NOT 두 개를 사용했다. clk를 NOT1로 뒤집어 master에 넣고, 그것을 NOT2로 한 번 더 뒤집어 slave에 넣었다. VHDL_2 p.12처럼 master에 clk를 바로 넣으면 하강 에지에서 Q가 바뀌는데, 과제는 상승 에지이고 VHDL도 rising_edge를 쓰기 때문에 master 쪽에 NOT을 하나 더 두어 같은 동작이 되도록 했다.

처음에는 되먹임 회로라서 Q가 E(에러)로 나왔다. 공지의 방법대로 CLR 입력을 만들어 G4와 G8(Q′ 쪽 교차 NAND)의 세 번째 입력에 연결했다. J=K=0, clk=0에서 CLR을 0으로 내렸다가 1로 올리면 Q=0으로 정해지고, 그 뒤에는 CLR=1로 두고 테스트하였다. VHDL에서 q_i의 초기값을 '0'으로 준 것과 같은 역할이다.

### (1) Logisim 게이트 회로

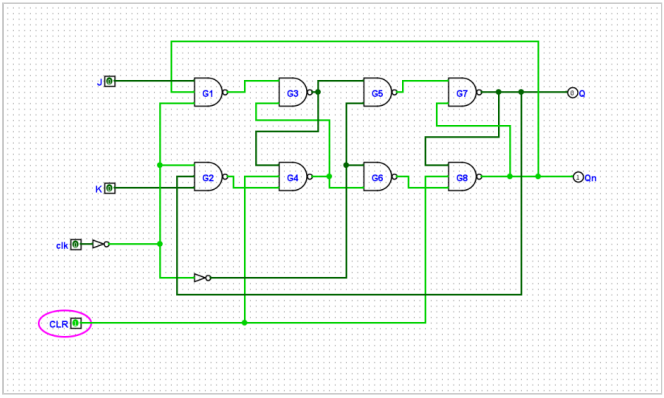



그림 35. CLR 0→1로 초기화 (Q=0)

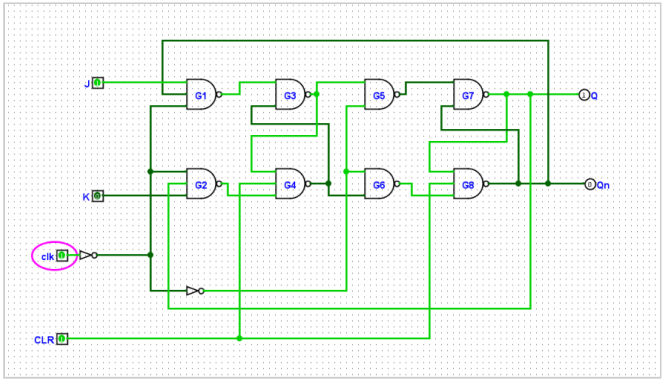



그림 36. Set (J=1, K=0)

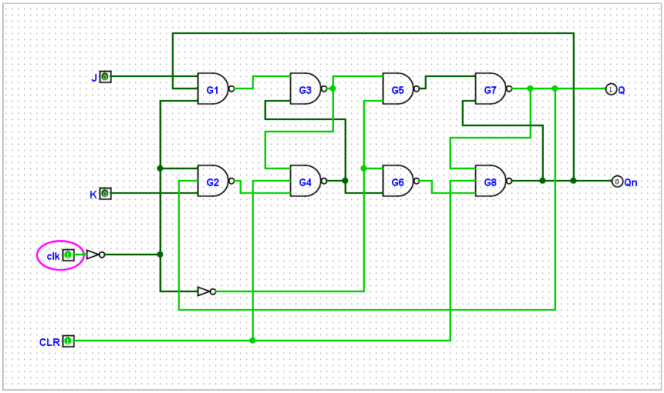



그림 37. Hold (J=K=0)

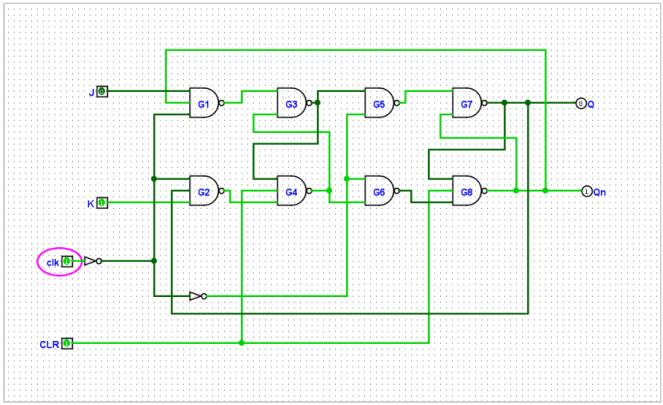



그림 38. Reset (J=0, K=1)

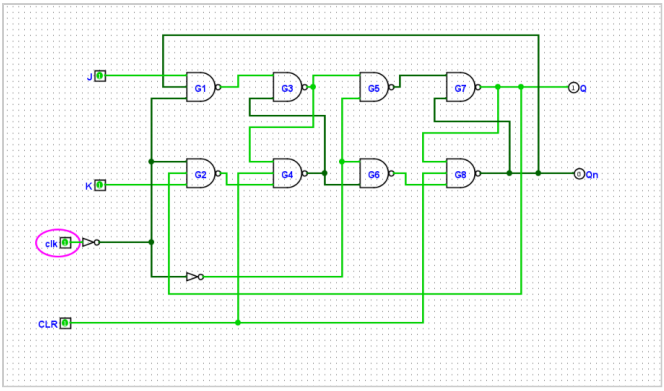



그림 39. Toggle 1회 (J=K=1)

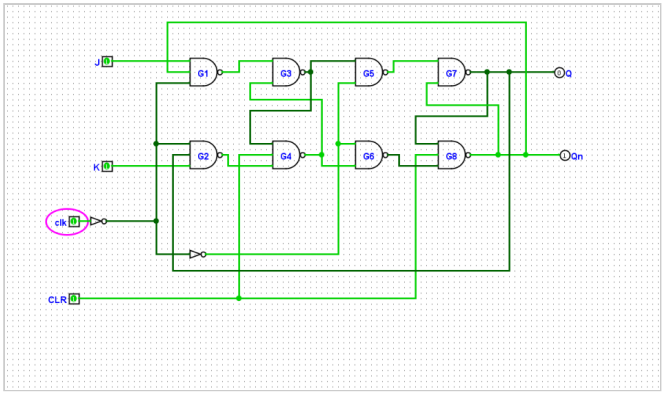



그림 40. Toggle 2회 (J=K=1)

### (2) VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- Positive-edge-triggered JK flip-flop
--   JK = 00 Hold, 01 Reset, 10 Set, 11 Toggle
entity jk_ff is
    port(
        j, k, clk : in  std_logic;
        q, qn     : out std_logic
    );
end entity jk_ff;

architecture bhv of jk_ff is
    signal q_i : std_logic := '0';
    signal jk  : std_logic_vector(1 downto 0);
begin
    jk <= j & k;                -- combine J and K into 2 bits

    process(clk)
    begin
        if rising_edge(clk) then
            case jk is
                when "00"   => q_i <= q_i;       -- Hold
                when "01"   => q_i <= '0';       -- Reset
                when "10"   => q_i <= '1';       -- Set
                when "11"   => q_i <= not q_i;   -- Toggle
                when others => q_i <= q_i;
            end case;
        end if;
    end process;

    q  <= q_i;
    qn <= not q_i;
end architecture bhv;
```

j와 k를 j & k로 묶어서 case문으로 4가지 경우를 나누었다. 토글할 때 현재 값을 읽어야 해서 내부 신호 q_i를 사용하였다.

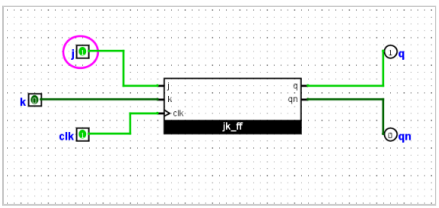



그림 41. Set (J=1, K=0) (VHDL)

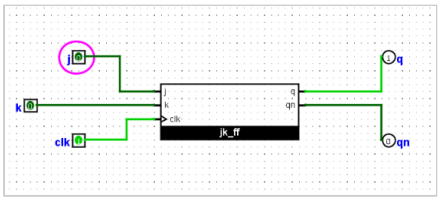



그림 42. Hold (J=K=0) (VHDL)

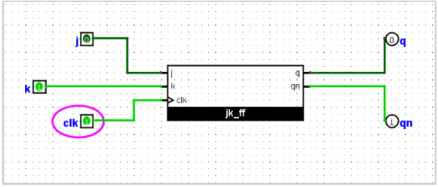



그림 43. Reset (J=0, K=1) (VHDL)

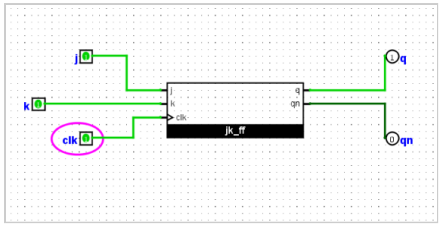



그림 44. Toggle 1회 (J=K=1) (VHDL)

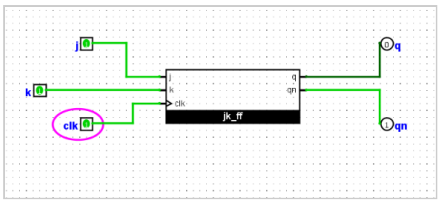



그림 45. Toggle 2회 (J=K=1) (VHDL)

### (3) 합성 결과

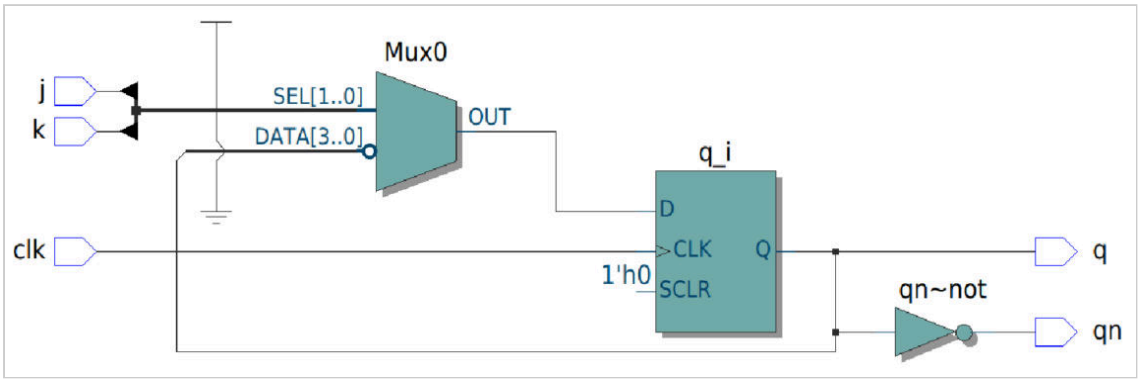



그림 46. p4 RTL Viewer 회로도

### (4) 시뮬레이션 결과

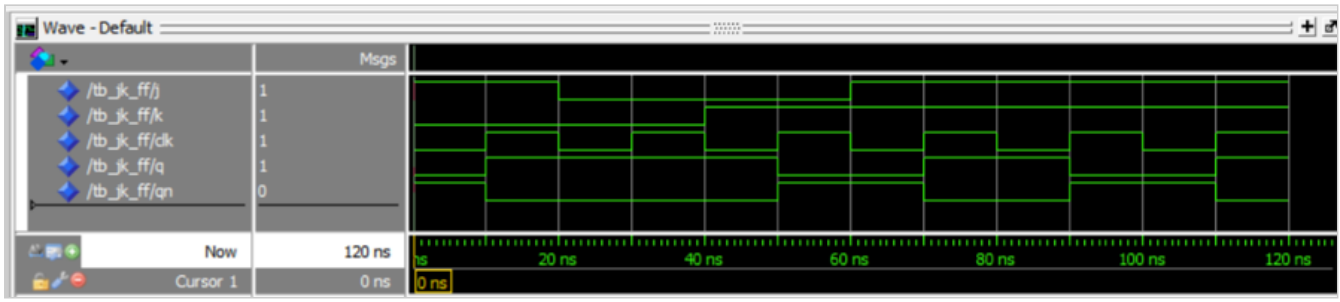



그림 47. tb_jk_ff 시뮬레이션 파형 (Questa)

10 ns에 Set, 30 ns에 유지, 50 ns에 Reset이 되고 70 ns부터는 J=K=1이라 에지마다 q가 뒤집혔다. 합성 회로도에서는 (j, k)를 선택 신호로 하는 4:1 MUX가 DFF의 D 입력 앞에 붙어 있다. MUX 입력이 Q, 0, 1, Q'이라서 case문 구조가 그대로 MUX가 된 것을 확인할 수 있었다.

## 5. set/clr가 있는 D 플립플롭

수업자료와 실습자료에는 set/clr가 있는 플립플롭의 게이트 회로가 없어서, 3번의 master-slave D 플립플롭을 그대로 가져오고 비동기 입력만 추가하였다. 공지에서 CLR을 교차 결합 NAND의 세 번째 입력에 넣었던 것과 같은 방법으로, 네 개의 교차 결합 NAND(G3, G4, G7, G8)를 3입력으로 바꾸었다.

- clr_n = clr′ (NOT 게이트) → G4, G8 (Q′ 쪽)
- pre_n = (set · clr′)′ (NAND 게이트 G9) → G3, G7 (Q 쪽)

clr=1이면 clr_n=0이 되어 G4, G8이 1이 되고 Q=0이 된다. 이때 pre_n은 set과 관계없이 1이 되므로 set이 들어가지 않는다. 그래서 clr가 set보다 우선이다. set=1, clr=0이면 pre_n=0이 되어 G3, G7이 1이 되고 Q=1이 된다. master와 slave를 둘 다 바꾸기 때문에 clk가 0이든 1이든 바로 적용되고, set/clr를 내린 뒤에도 값이 유지된다.

| clr | set | clk | Q+ |
|---|---|---|---|
| 1 | x | x | 0 |
| 0 | 1 | x | 1 |
| 0 | 0 | 상승 에지 | D |
| 0 | 0 | 그 외 | Q |

### (1) Logisim 게이트 회로

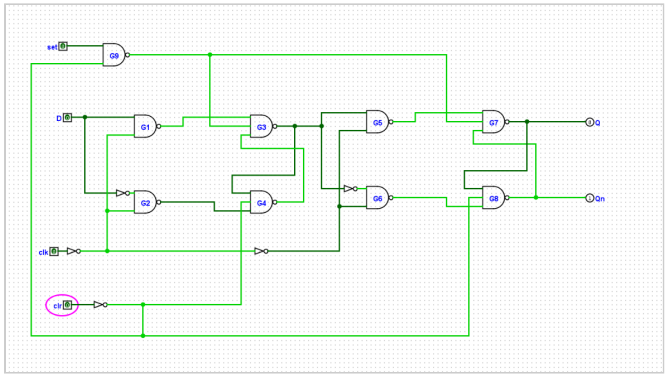


그림 48. 초기 상태

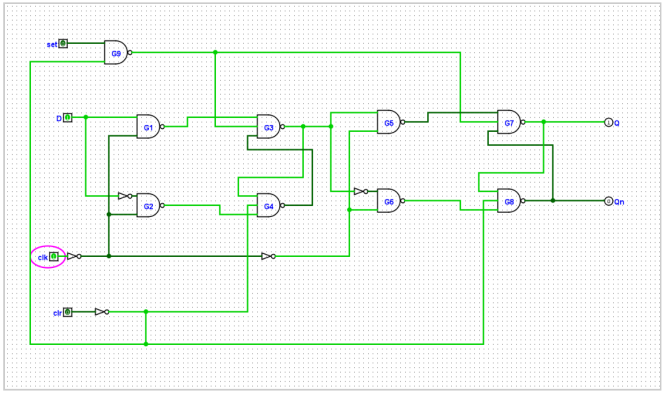


그림 49. 일반 동작, D=1

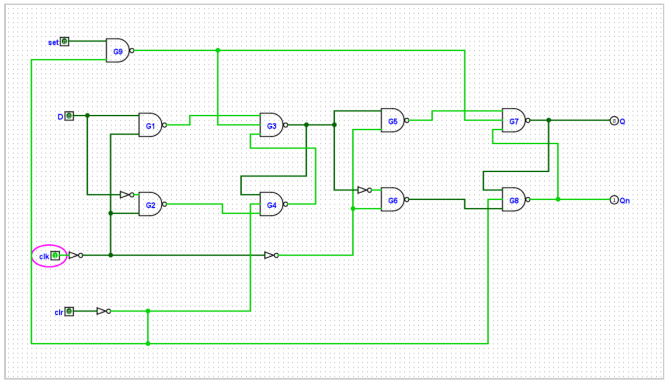


그림 50. 일반 동작, D=0

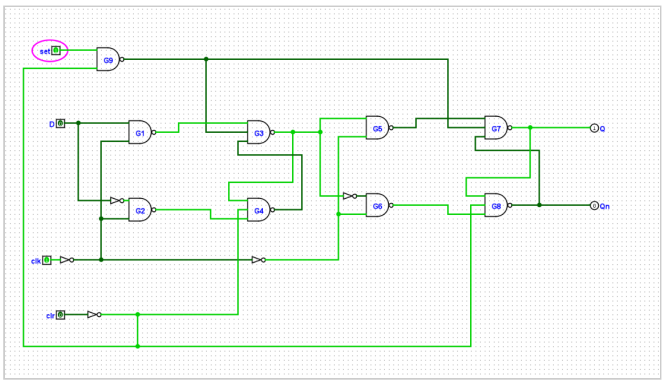


그림 51. 비동기 set

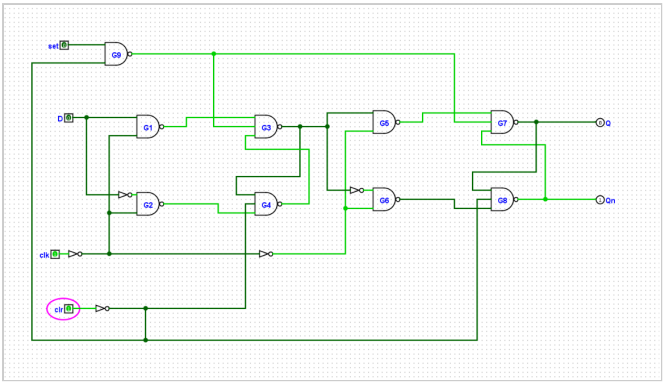


그림 52. 비동기 clr

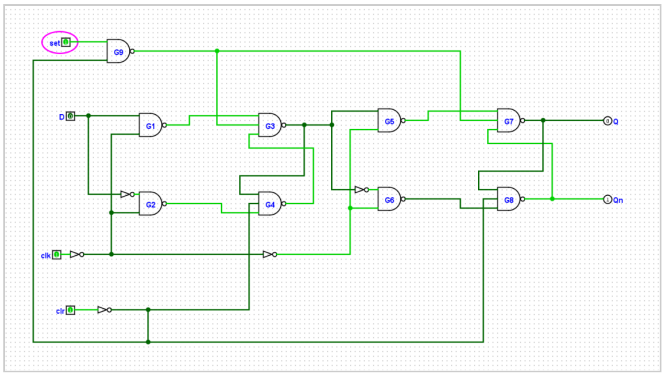


그림 53. set=clr=1 (clr 우선)

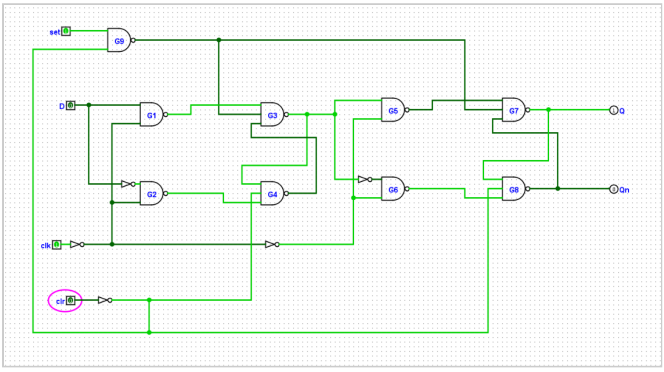


그림 54. clr 해제 후 set 적용

처음에는 slave 쪽(G7, G8)이 E로 나와서, clr를 1로 올렸다가 0으로 내려 초기화한 뒤 테스트하였다.

### (2) VHDL 코드

```
library ieee;
use ieee.std_logic_1164.all;

-- Positive-edge D flip-flop with asynchronous set / clear
--   clr = '1'             : Q <= '0' immediately (highest priority)
--   set = '1' (clr = '0') : Q <= '1' immediately
--   otherwise             : normal D flip-flop
entity d_ff_sc is
    port(
        d, clk   : in  std_logic;
        set, clr : in  std_logic;
        q, qn    : out std_logic
    );
end entity d_ff_sc;

architecture bhv of d_ff_sc is
    signal q_i : std_logic := '0';
begin
    process(clk, set, clr)
    begin
        if clr = '1' then              -- checked first -> clear has priority
            q_i <= '0';
        elsif set = '1' then
            q_i <= '1';
        elsif rising_edge(clk) then
            q_i <= d;
        end if;
    end process;

    q  <= q_i;
    qn <= not q_i;
end architecture bhv;
```

감도 목록에 clk, set, clr를 모두 넣어서 클럭 에지가 없어도 set, clr에 바로 반응하게 했다. if문에서 clr를 제일 먼저 검사하기 때문에 둘 다 1이면 clr가 적용된다. 실습자료(VHDL_2 p.10)의 코드와 같은 구조이다.

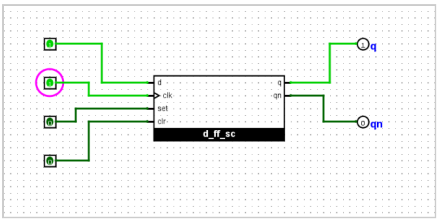


그림 55. 일반 동작, D=1 (VHDL)

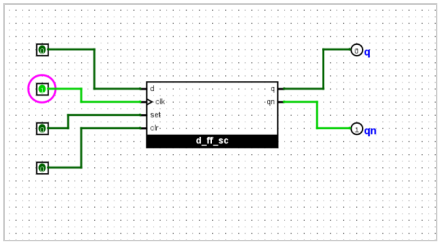


그림 56. 일반 동작, D=0 (VHDL)

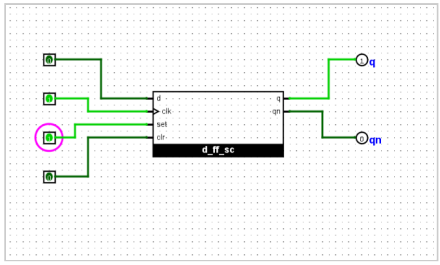


그림 57. 비동기 set (VHDL)

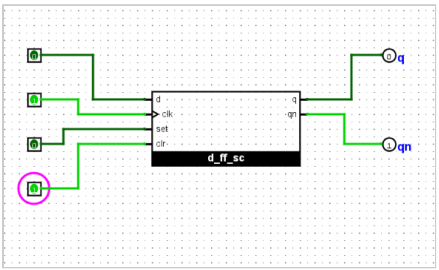


그림 58. 비동기 clr (VHDL)

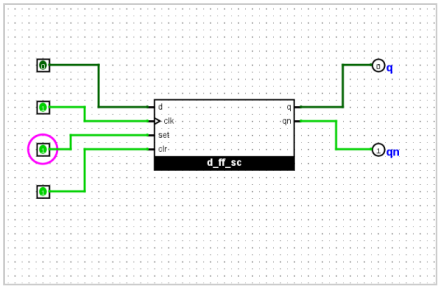


그림 59. set=clr=1 (clr 우선) (VHDL)

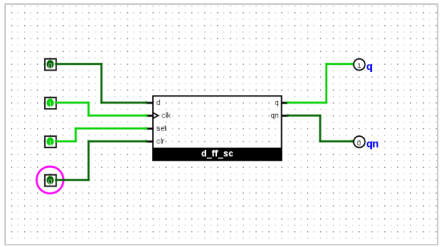


그림 60. clr 해제 후 set 적용 (VHDL)

### (3) 합성 결과

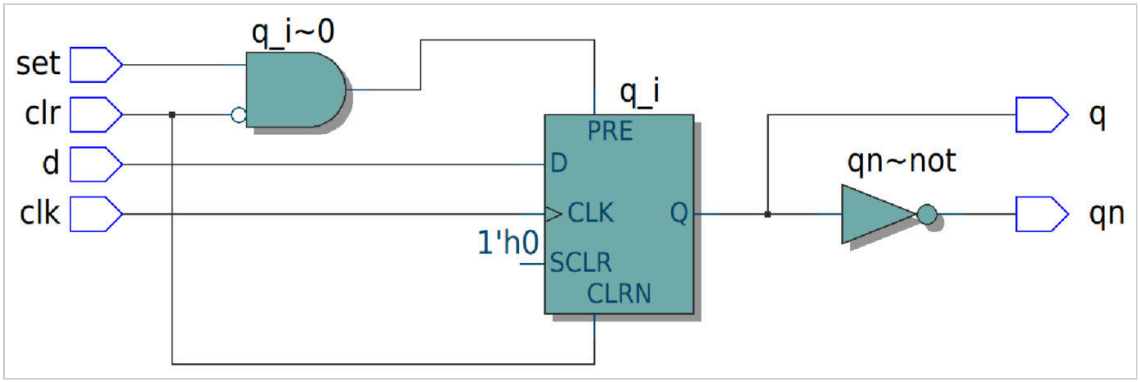



그림 61. p5 RTL Viewer 회로도

### (4) 시뮬레이션 결과

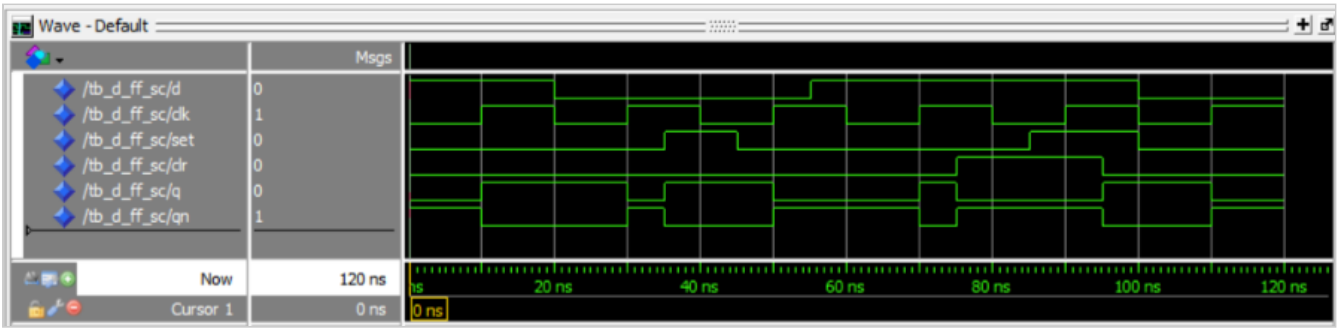



그림 62. tb_d_ff_sc 시뮬레이션 파형 (Questa)

10 ns, 30 ns 에지에서는 일반 D 플립플롭처럼 동작하고, 35 ns에 set=1을 넣으면 에지가 없어도 바로 q=1이 되었다. 75 ns에 clr=1을 넣으면 바로 q=0이 되고, 85 ns에 set까지 1로 만들어도 q=0이 유지되었다(clr 우선). 95 ns에 clr를 내리면 set이 적용되어 q=1이 되었다.

합성 회로도에서 clr는 DFF의 CLRN에 연결되고 set은 AND(set·clr′)를 거쳐 PRE로 들어간다. 이 AND 게이트가 clr 우선순위를 만드는 부분이고, 게이트 회로에서 pre_n = (set·clr′)′로 만든 것과 같은 논리이다.

## 6. 수업자료 회로와 다르게 구현한 부분

게이트 회로는 수업자료(9-1lec), 실습자료(VHDL_2), 11/11 공지에 나온 회로를 그대로 따르는 것을 원칙으로 했다. 다만 과제 요구사항과 맞지 않거나 원본이 없는 부분은 아래처럼 바꾸었다. 바꾼 부분은 모두 Logisim에서 다시 시뮬레이션해서 확인하였다(문제별 확인 항목 수: p1 6개, p2 5개, p3 7개, p4 9개, p5 14개, 모두 기대값과 일치).

| 문제 | 원본 (출처) | 이번 구현 | 바꾼 이유 |
|---|---|---|---|
| 1 | SR 래치를 NAND로 구현, 입력 NS·NR은 0일 때 동작 (VHDL_2 p.2) | NOR 2개 교차 결합 (9-1lec p.9 회로) | 과제 1번이 "NOR gates"로 지정함 |
| 2 | enable 입력 이름이 CLK (VHDL_2 p.3~6) | 입력 이름 en, 회로 구조는 동일 | 과제 2번이 en으로 지정함 |
| 3, 4, 5 (VHDL) | clk'event and clk='1' (VHDL_2 p.9, p.10, p.13) | rising_edge(clk) | 과제 3번이 rising_edge 사용을 지정함. 0→1 변화에서는 같은 동작 |
| 4 (클럭) | master에 CLK, slave에 NOT CLK (VHDL_2 p.12, 11/11 공지) | CLK→NOT→master, 그 출력을 다시 NOT→slave | 원본대로 하면 하강 에지 동작이 됨. 과제는 상승 에지 |
| 4 (초기화) | CLR을 C 게이트에만 연결 (11/11 공지, 래치 1개) | CLR을 G4(C)와 G8 두 곳에 연결 | 래치가 2개로 늘어서 둘 다 초기값이 필요함 |
| 4 (입력단) | AND 게이트 + SR 래치 (VHDL_2 p.12) | 3입력 NAND (11/11 공지, VHDL_2 p.11 형태) | 공지에서 쓴 형태를 그대로 사용. AND+NAND를 합친 것과 같음 |
| 5 (게이트) | 게이트 회로 없음 (VHDL_2 p.10은 VHDL만 있음) | 3번 회로 + 교차 NAND 3번째 입력에 pre_n, clr_n | 원본이 없어서 수업자료 회로와 공지 방법을 조합함 |
| 2, 3, 4, 5 | 과제 문구는 "NAND gates", "an inverter" | NOT 게이트 사용 (래치 안 D 반전, 클럭 반전) | VHDL_2 p.3, p.8과 9-1lec p.16, p.20 그림에 NOT이 들어 있음 |

### (1) 문제 1: NOR로 구현한 이유

실습자료 VHDL_2 p.2의 SR 래치는 NAND 두 개로 되어 있고 입력이 0일 때 동작하는(NS, NR) 형태이다. 하지만 과제 1번은 "Build the circuit using NOR gates"라고 되어 있어서 수업자료 9-1lec p.9의 NOR SR 래치를 그대로 사용했다. 9-1lec p.12에 나온 것처럼 NAND 래치는 NOR 래치의 입력과 출력을 뒤집은 것이라 기능은 같다. 입력이 1일 때 동작하므로 과제에 적힌 Set/Reset/Hold/금지 입력 표와 바로 맞는다.

### (2) VHDL에서 rising_edge를 쓴 이유

실습자료의 VHDL 코드는 클럭 에지를 clk'event and clk='1'로 확인한다. 과제 3번에 "using rising_edge(clk)"라고 적혀 있어서 3번에 rising_edge를 썼고, 같은 방식으로 4번, 5번도 맞추었다. 두 방법은 clk가 0에서 1로 바뀔 때 똑같이 참이 되므로 이번 테스트벤치에서는 결과가 같다. rising_edge는 U나 X에서 1로 바뀌는 경우는 에지로 보지 않는다는 점만 다르다. 실습자료 코드와 달리 qn 출력을 추가했는데, 과제에서 Q와 Q′를 모두 확인해야 해서 넣었다. 4번 VHDL은 실습자료 p.13 코드에 있는 clr, set 입력을 넣지 않았다. 과제 4번은 J, K 네 가지 동작만 요구하기 때문이다.

### (3) 문제 4: master 앞에 NOT을 둔 이유

11/11 공지와 VHDL_2 p.12는 master 래치에 CLK를 그대로 넣고 slave에 반전된 CLK를 넣는다. 이렇게 하면 CLK=1일 때 master가 입력을 받고 CLK가 1→0으로 떨어질 때 slave로 값이 넘어가서, Q는 하강 에지에서 바뀐다. 과제 4번은 "positive-edge-triggered JK Flip-Flop"이고, VHDL도 rising_edge를 쓰며, 두 구현이 같은 논리

기능이어야 한다는 조건이 있다. 그래서 9-1lec p.20 그림처럼 반전 클럭을 master 쪽에 넣었다. 그림 아래에도 master-slave는 결국 상승 에지로 동작한다고 적혀 있다. 배치는 3번(VHDL_2 p.8)과 똑같이 했다. CLK를 NOT 하나로 뒤집어 master에 넣고, 그것을 한 번 더 뒤집어 slave에 넣었다. slave가 master의 반전 클럭을 받는다는 점은 공지와 같다. Logisim에서 확인한 결과, Q는 clk가 0→1로 바뀔 때만 변했고 1→0일 때는 변하지 않았다. clk=0일 때 J, K를 바꿔도 Q는 그대로였다.

### (4) 문제 4: CLR 입력

공지 회로는 래치가 하나라서 C 게이트 하나에만 CLR을 넣었다. 이번 회로는 master와 slave 두 래치로 되어 있어서 같은 위치인 Q′ 쪽 교차 NAND 두 개(G4, G8)에 CLR을 넣었다. slave에 넣지 않으면 clk=0 동안 slave가 E 상태로 남는다. CLR은 Logisim에서 처음에 값이 정해지지 않는 문제를 풀기 위한 입력이다. 처음에 0→1로 한 번 넣은 뒤에는 1로 두기 때문에 JK 동작에는 영향이 없다. VHDL에는 이 입력이 없고, 대신 signal q_i : std_logic := '0'으로 초기값을 준다. 따라서 CLR=1인 정상 동작에서 게이트 회로와 VHDL은 같은 기능이다.

### (5) 문제 5: 게이트 회로를 만든 방법

수업자료와 실습자료에는 set/clr가 있는 플립플롭의 게이트 회로가 없고, VHDL_2 p.10에 VHDL 코드만 있다. 그래서 새로운 구조를 쓰지 않고 이미 수업자료에 있는 부분만 조합했다. 기본 회로는 3번의 master-slave D 플립플롭(VHDL_2 p.8)이다. 비동기 입력은 11/11 공지에서 CLR을 교차 결합 NAND의 세 번째 입력에 넣은 방법을 그대로 써서 G3, G4, G7, G8을 3입력으로 바꾸었다. NAND 입력에 0이 들어가면 출력이 무조건 1이 되기 때문에 클럭과 상관없이 바로 값이 정해진다. clr 우선순위는 pre_n = (set·clr′)′로 만들었다. clr=1이면 set 값과 관계없이 pre_n=1이 되어 set이 막힌다. 이것은 VHDL_2 p.10 코드에서 clr를 먼저 검사하는 것과 같은 의미이고, Quartus 합성 결과에서도 set 앞에 AND(set·clr′)가 생겼다.

### (6) NOT 게이트를 사용한 이유

과제 2번과 4번에는 "NAND gates", 3번에는 "two D Latches and an inverter"라고 적혀 있다. 하지만 기준으로 삼은 그림인 VHDL_2 p.3의 D 래치와 9-1lec p.16에는 D를 뒤집는 NOT이 있고, VHDL_2 p.8의 D 플립플롭에는 클럭용 NOT이 두 개 있다. 수업자료 그림을 그대로 따르기 위해 NOT 게이트를 사용했다. NOT은 NAND의 두 입력을 묶은 것과 같아서 NAND만 쓴 회로와 기능 차이는 없다. 3번 회로는 "D 래치 2개"와 "클럭 반전"이라는 구성은 과제와 같고, 클럭 반전을 VHDL_2 p.8처럼 NOT 두 개로 나눈 것만 다르다.

## 7. 결론

다섯 문제 모두 Logisim 게이트 회로, Logisim VHDL 시트, Questa 파형의 결과가 같게 나왔다. 따라서 게이트로 만든 회로와 VHDL 코드가 같은 동작을 한다는 것을 확인하였다.

래치(1, 2번)는 입력 레벨에 따라 바로 바뀌고, 플립플롭(3~5번)은 클럭 상승 에지에서만 바뀐다. 래치 두 개를 master-slave로 이어서 에지 동작이 만들어지는 과정을 직접 확인할 수 있었다. 또 Quartus 합성 결과를 보면서 VHDL 문장이 어떤 하드웨어가 되는지 알 수 있었다. else 없는 if는 래치가, rising_edge는 DFF가, case문은 MUX가 되었고, if문 순서로 정한 우선순위는 AND 게이트로 나타났다.

제출 파일: p1~p5 .vhd, 테스트벤치(tb_*.vhd), assignment3.circ, Quartus 프로젝트(quartus/a3.qpf, a3.qsf), 캡처 폴더(captures/), 보고서 PDF